

BEL C22 · JAVA TRACK

Java Revision Study Guide

Condensed notes, code snippets, mental models & memory tricks for every topic covered so far — one doc instead of nine files.

- 01 JDK, JRE & JVM
- 02 Abstraction
- 03 Design Principles — the big picture
- 04 DRY, KISS & YAGNI
- 05–09 SOLID — SRP, OCP, LSP, ISP, DIP

01 JDK, JRE & JVM

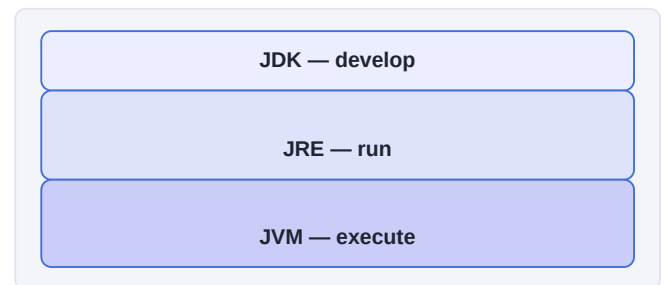
IN SHORT

- **JDK** = build & run. Has javac, javadoc, jar + the JRE inside it. What developers install.
- **JRE** = run only. JVM + core libraries (java.lang, java.util...). No compiler.
- **JVM** = executes bytecode. Gives platform independence — same .class file runs anywhere.
- Containment: **JDK** $\supset$ **JRE** $\supset$ **JVM** — each wraps the next.

WHO NEEDS WHAT

Role	Needs	Why
Developer	JDK	write + compile code
End user	JRE	just run the app

MENTAL MODEL — NESTING DOLLS



Trick: remember the verbs in order — **D**evelop (JDK) → **R**un (JRE) → **E**xecute (JVM). Say "**D-R-E**, biggest to smallest."

02 Abstraction

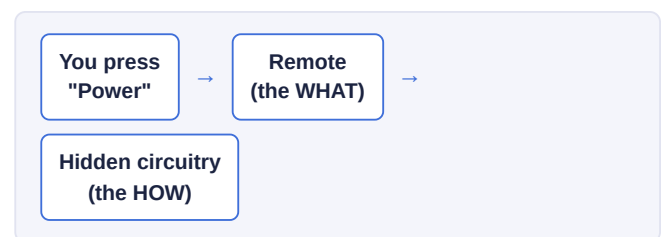
IN SHORT

- Hide the **how**, expose only the **what**. Reduces complexity, separates interface from implementation.
- Two ways to achieve it in Java: **abstract classes** (0–100% abstraction) and **interfaces** (100%).
- Abstract class: can mix abstract + concrete methods, has fields & constructors, but **cannot be instantiated** directly.
- Use it when subclasses share *some* behavior but differ in specifics (e.g. all shapes have color, area differs).

SNIPPET

```
abstract class Shape {  
    abstract double area();    // no body – must  
    override  
    void describe() {          // concrete –  
        inherited as-is  
        System.out.println("I am a shape.");  
    }  
}
```

MENTAL MODEL — THE TV REMOTE



You never see infrared signals or circuit boards — just the button. That's abstraction.

Trick: before writing a class, ask "does the *caller* need to know HOW this works, or just WHAT it does?" If just WHAT → abstract it.

03 Design Principles — the big picture

IN SHORT

- Guidelines for code that is **maintainable, scalable, reusable, robust**.
- Coming up: **DRY, KISS, YAGNI**, the 5 **SOLID** principles, and composition-over-inheritance.

PRINCIPLES VS. PATTERNS

	Principles	Patterns
Scope	abstract guideline	concrete solution
Example	"one responsibility"	"use Observer"

MENTAL MODEL — COMPASS VS. MAP

Principles
= compass
(direction)

Patterns
= map route
(specific path)

Trick: principles tell you *what to aim for*; patterns show you *how to get there*. Never confuse a rule of thumb with a recipe.

04 DRY, KISS & YAGNI

IN SHORT

Principle	Rule	Ask yourself
DRY	Don't Repeat Yourself — one source of truth per bit of logic	"Have I written this before?"
KISS	Keep It Simple — prefer the plain solution	"Can a junior read this in 10s?"
YAGNI	You Aren't Gonna Need It — build only what's needed now	"Does anyone need this <i>today</i> ?"

SNIPPET — KISS IN ACTION

```
// nested (bad)           // guard clauses
(good)
if (x != null) {
  return;
  if (x > 0) {
    if (isPaid) { work(); }
  }
  return;
}

if (x == null)
  if (x <= 0) return;
  if (!isPaid)
    work();
```

MENTAL MODEL — THREE GUARDRAILS

DRY
no duplicate logic

KISS
no needless complexity

YAGNI
no speculative code

Analogy: updating your phone number in **one** contacts app (DRY) beats editing 50 apps by hand.

Trick: mnemonic — "**D**on't copy, **K**eep calm, **Y**ou ain't need it yet." Violations: copy-pasted tax logic (DRY), 6-level if-else (KISS), a `multiply()` nobody asked for (YAGNI).

SOLID — five principles for maintainable OOP

S Single Responsibility Principle

IN SHORT

- "A class should have only **one reason to change**." — Uncle Bob
- One responsibility → fewer tests, lower coupling, changes stay contained.

SNIPPET

```
// bad: 3 reasons to change
class OrderService {
  placeOrder(); calculateTotal(); sendEmail();
}
// good: split by responsibility
class OrderService { placeOrder(); }
class OrderCalculator { calculateTotal(); }
class EmailService { sendEmail(); }
```

MENTAL MODEL — THE RESTAURANT

Chef
cooks

Waiter
serves

Cashier
bills

Trick: describe the class in one sentence. If the sentence needs "**AND**", split it. "Manages employees **AND** sends emails" = violation.

O Open/Closed Principle

IN SHORT

- Open for **extension**, closed for **modification**. Add features via new code, not edits to old code.
- Kills the "growing if-else chain" anti-pattern.

SNIPPET

```
// bad: edit this method for every new shape
if (type.equals("circle")) drawCircle();
else if (type.equals("rect")) drawRectangle();

// good: new shape = new class, nothing else
// touched
interface Shape { void draw(); }
class Triangle implements Shape { draw() {...} }
```

MENTAL MODEL — THE USB PORT

USB port
(never rewired)

← keyboard / mouse /
phone (plug in new)

Trick: "New variant? New class. Done." If you're reopening a working class to add a branch, you're breaking OCP.

L Liskov Substitution Principle

IN SHORT

- Any subclass must be usable wherever the parent is expected — **no surprises**.
- Classic break: `Ostrich` extends `Bird` but `fly()` throws.

SNIPPET

```
// bad
class Ostrich extends Bird {
    fly() { throw new
    UnsupportedOperationException(); }
}

// good - redesign the contract
abstract class Bird { abstract void move(); }
class Sparrow extends Bird { move() { fly(); } }
class Ostrich extends Bird { move() { walk(); } }
```

MENTAL MODEL — THE SWAP TEST

Code expects
Parent



Swap in
any Child



Still works?
Must be YES

Trick: hire a "driver" who says "can't drive manual" — contract broken. Red flags: child throws, child does nothing, child adds surprise preconditions.

I Interface Segregation Principle

IN SHORT

- Don't force classes to depend on methods they don't use. Many **small** interfaces > one fat one.
- ISP violations often **cause** LSP violations (forced empty/throwing methods).

SNIPPET — FROM ISPDEMO.JAVA

```
interface Workable { void work(); }
interface Eatable { void eat(); }

class RobotWorker implements Workable {
    work() { /* only what a robot needs */ }
    // no eat() - not forced to fake it
}
```

MENTAL MODEL — THE MENU

One 50-item
menu (fat)



Veg menu /
Non-veg menu

Trick: spot it by the smell — a method body that's just throw new `UnsupportedOperationException()` or a suspicious `// doesn't apply here` comment.

D Dependency Inversion Principle

IN SHORT

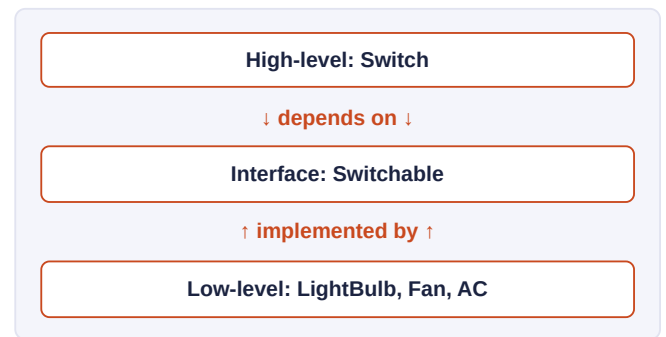
- High-level modules shouldn't depend on low-level ones directly — **both** depend on an abstraction.
- Enables **dependency injection**: pass dependencies in (constructor/setter) instead of creating them with new inside.

SNIPPET

```
// bad - hardcoded, tightly coupled
class Switch { LightBulb bulb = new LightBulb();
}

// good - depends on the interface
interface Switchable { void turnOn(); }
class Switch {
    Switch(Switchable device) { this.device = device; }
}
```

MENTAL MODEL — INVERSION



Trick: USB-C charger — your phone depends on the *standard*, not the brand. "Depend on the plug shape, not the plug brand."